Handwritten Digit Recognition & Classification

1st Varun Amitabh
Electrical and Computer Engineering
University of Florida
Gainesville, USA
amitabh.varun@ufl.edu

2nd Shivali Patel
Electrical and Computer Engineering
University of Florida
Gainesville, USA
spatel30@ufl.edu

3rd Jorge Fonrodona
Electrical and Computer Engineering
University of Florida
Gainesville, USA
jfonrodona@ufl.edu

4th Karan Chauhan
Electrical and Computer Engineering
University of Florida
Gainesville, USA
chauhan.k@ufl.edu

***Abstract*—Handwritten digit detection is one of the most fundamental problems in Computer Vision and Machine Learning due to its practicality and complexity. However, there are numerous ways in which this can be accomplished, and finding a model that yields the most accurate results is constantly a subject of debate. This paper discusses the method and result of using advanced deep learning models to deal with the handwritten digit detection problem. 4190 images with single, double and triple sequence were used as the training dataset. The detection algorithm includes data preprocessing, data augmentation, and training a model through YOLO library. Experiments were carried out to evaluate model performance comparing accuracy, and precision. The effectiveness of the results is discussed.**

***Index Terms*—handwritten digit detection, object detection, YOLO, OpenCV**

I. Introduction

Handwritten digit detection continues to be an important topic of research in computer vision and pattern recognition, owing to its widespread application in industries like automated data input, document processing, and postal systems. Because handwriting styles, stroke abnormalities, and backdrops vary widely in real-world situations, the problem remains challenging even after all these years. Strong detection algorithms and effective preprocessing processes must be integrated to handle these complications.

Handcrafted feature extraction techniques were a major component of traditional handwritten digit detection algorithms. But with real-world datasets, these methods frequently have trouble with scalability and generalization. With convolutional neural networks (CNNs), exhibiting state-of-the-art performance on common datasets, the possibility for managing such unpredictability has been greatly increased with the introduction of deep learning. Furthermore, complicated jobs requiring real-time detection and recognition have showed promise when tackled by sophisticated object detection frameworks like You Only Look Once (YOLO).

Recent research has investigated numerous strategies for improving handwritten digit identification with modern machine learning algorithms. For instance, the authors of [1] used deep learning methods to identify handwritten characters while maximizing computing efficiency and detection accuracy. Motivated by this work, the current study builds a thorough detection pipeline for single, double, and triple-digit sequences using YOLO and the OpenCV library. To improve the model's accuracy and resilience, this pipeline includes thorough preprocessing, augmentation, and training stages.

This study's main contributions include the development of a scalable and efficient model capable of generalizing across different handwriting styles, as well as a thorough evaluation of its performance on a training dataset of 4,190 photos.

II. Data Preprocessing

The preprocessing and augmentation steps involved in the code serve to enhance the quality and diversity of the dataset with a view to improving performance in handwritten digit recognition with YOLOv8. The preprocessing steps are applied to clean and prepare the raw images for better recognition by the model.

Firstly, the images were resized to a fixed resolution of 224×224 to ensure uniform input dimensions for the model, reducing computational complexity and aligning with YOLO's input requirements. Another reason to include this in the preprocessing step was the fact that since the images were so raw, containing numerous misclassifications in annotations and incorrect bounding boxes, we wanted to at least maintain some sort of uniformity in the dataset.

Second, the images were converted to grayscale. The thought behind this was that digits are henerally monochrome and removing color channels would reduce noise and dimensionality without losing relevant features.

This was followed by a Gaussian blur, applied to smoothen the images and reduce noise. This was done to eliminates high-frequency noise, such as smudges, digit leakage or specks, which were seen in the dataset images.

Since the images were grayscale, Contrast Limited Adaptive Histogram Equalization (CLAHE) technique was applied to enhance contrast locally within the image. The thought was to improve the visibility of faint or light strokes, which would

be useful for ensuring consistent digit clarity across varying image conditions.

Next, adaptive Gaussian thresholding was applied to convert images to binary format. This would mainly emphasize digit outlines and suppresses irrelevant background pixels, enhancing the separability of digits from the background. These steps were later included as part of the preprocessing as on having intially trained the model and seen the bounding boxes of gthe predicted model on training set, we could visually make out that the model was struggling to capture the digits with unclear backgrounds or faint strokes of digits in the images.

Since some images in the dataset were in the corners or not completely visible, edge detection via the Canny method was applied. It essentially highlights the structure of digits, making features more distinct for recognition.

Skew Correction was also applied to ensures proper alignment of digits, which is useful for accurate bounding box detection and classification.

Lastly, Normalization of pixel values to a range of [0,1] was done in an attempt to improve model stability and training convergence.

- Dataset Preparation
  Images and label files in the YOLO format make up the dataset used in the research. Images are supplied in.png format, and labels specify bounding box coordinates and class IDs that are normalized to the image's size. The dataset is divided into training and validation sets in an 80:20 ratio to guarantee that there is enough data for both model evaluation and training.
- Data Augmentation
  Augmentation techniques are used to improve model generalization and increase the diversity of the training data. The pictures are subjected to random rotation. By increasing transformation invariance, augmentation not only broadens the diversity of the training dataset but also improves network robustness and generalization. Bounding box coordinates are updated following rotation to ensure accuracy. Motivated by recent developments in loss function optimization and bounding box regression [4], the new coordinates are adapted to the size of the rotated image by applying the rotation matrix to the center of each bounding box. To prevent mistakes caused by out-of-frame coordinates, bounds are enforced.
  When images are rotated, the bounding box coordinate of the image also changes, hence we perform coordinate transformation to update and match the new position of the objects in the rotated images.
- YOLO-Compatible Dataset Generation
  A directory structure that is compatible with YOLO is used to store the transformed images together with their updated labels. The subsets for training and validation are each given its own folder. Labels are saved as .txt files in a corresponding labels folder, while images are kept in a dedicated images folder. The class ID, bounding box width, height, and center coordinates are all included in each label file, as is required for YOLO dataset labels.

A. Model Training

A YOLO-based object detection model is trained using the preprocessed dataset. YOLO was chosen because to its effectiveness in multi-object tracking and real-time detection. The generated datasets are used in the training process to identify patterns and efficiently recognize handwritten digits.

a) Advantages of the Proposed Approach: The pipeline that has been put in place guarantees excellent data preparation and augmentation, giving the model a varied and precise dataset. Performance in real-world situations is enhanced as a result, especially when it comes to identifying and locating handwritten numbers in intricate inputs. The system achieves precision and scalability by utilizing YOLO's sophisticated detecting capabilities.

III. Implementation

The handwritten digit detection system integrates the YOLO object detection framework with OpenCV to create a comprehensive pipeline for efficient data preparation and model training.

YOLOv8 is an architecture of streamlined networks integrating convolutional layers along with attention mechanisms, featuring higher feature extraction capabilities. Object detection, segmentation, and classification are supported within a unified framework. It provides pre-trained weights on large, diverse datasets such as COCO, which can be fine-tuned for specific tasks. Real-time detection makes it appropriate to use in applications where faster inference is required.

a) Motivation Behind Transfer Learning: Transfer learning takes a model that has been pre-trained on a large, generic dataset and fine-tunes it on a specific task or dataset. For our implementation:

To implement the code, first, run the preprocessing code and upload the output sets to HiPerGator to train the model. For the test set, preprocess the data, upload it, and use the best-trained model weights to make predictions on the test set.

- The handmade handwritten digit dataset is relatively small compared to datasets like COCO. Training a YOLOv8 model from scratch would likely lead to overfitting and suboptimal generalization.
- The pre-trained YOLOv8 weights already encode some low-level features like edges, textures, and patterns inherent in most image datasets; hence, these features will inherently be useful for the task of digit detection.
- Fine-tuning a pre-trained model can reduce the computational cost and time required for training by an order of magnitude compared to training from scratch.

b) Using Transfer Learning with YOLOv8: This differs from a simple classification task, wherein the model needs to find digits in different positions and orientations in every image. YOLOv8 is very good at localizing objects with bounding boxes, making it ideal for this dataset. Additionally, The handwritten digit dataset contains variability in: Size and shape of digits; Background noise and contrast; Orientation and skew of handwritten entries. YOLOv8 is resilient to this kind of

variability for accurate predictions. Moreover, the lightweight architecture of YOLOv8 allows it to train quickly and perform inference efficiently on resource-constrained hardware, making it a good choice in our view to use for training on the dataset.

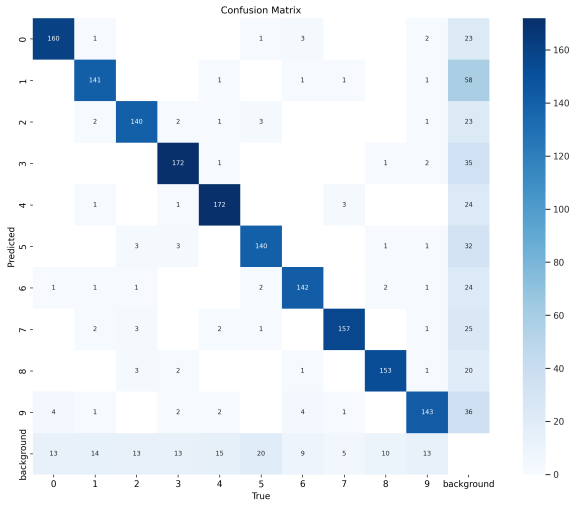


Fig. 1. Confusion Matrix of Best Performing Model

IV. Experimentation

The five experiments that were carried out to improve the handwritten digits recognition model using YOLOv8 are detailed in this section. Finding the best combination of hyperparameters and model settings was the aim of these experiments, with an emphasis on resolving bounding box prediction problems and overcoming overfitting. A thorough explanation of each experiment's objectives, conclusions, and how the outcomes influenced changes to the model's training pipeline can be found below.

- Experiment 1 The base configuration we adopted used the YOLOv8n architecture, with an image size of 640px, batch size of 32, and learning rate of 0.0001. Early stopping was employed with a patience of 10 epochs, and the training set was augmented. In total, the model ran for 50 epochs; that was done in order to find the balance between performance and not to overfit too much by early stopping and data augmentation.
  Findings: The performance metrics for this experiment were:
  mAP50: 0.57683 mAP50-95: 0.57573 Precision (B): 0.58862 Recall (B): 0.46133 Although early stopping and data augmentation were applied, the learning rate of 0.0001 was too low to allow the model to converge efficiently, as reflected in the relatively lower precision and recall values. There was some improvement in the model, in terms of bounding box prediction, compared to previous experiments, but the mislabeling and multiple bounding boxes around closely packed digits were still present.
- Experiment 2 In this, the YOLOv8n (nano) architecture was used with default training parameters in order to create a baseline model. With an image size of 640 pixels, a batch size of 8, an AdamW optimizer, and an initial learning rate of 0.001, the model was trained for 50 epochs.
  Results: The following metrics demonstrated the model's respectable initial performance: mAP50-95: 0.49323 mAP50: 0.5773. Bounding box predictions, however, had problems. The model frequently misinterpreted the digits or produced several bounding boxes around a single digit when the digits were positioned too closely together. Furthermore, the model had trouble handling digits with boxed or ruled backgrounds (such as those on pages), most likely as a result of inadequate segmentation or background interference.
  Initial findings showed that overfitting was taking place, especially when the digits were positioned closely together. In order to decrease these errors and enhance generalization, the training setting has to be changed.

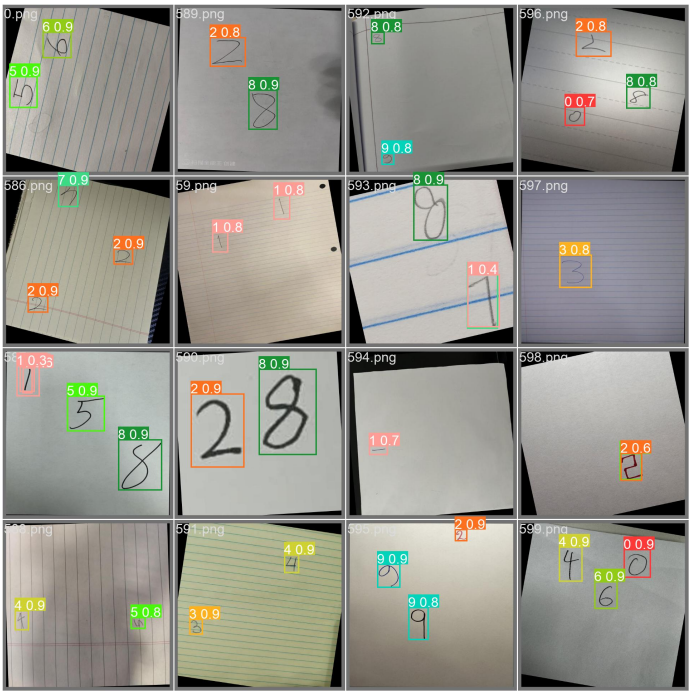


Fig. 2. Bounding Box Prediction of Best Performing Model on Images

- Experiment 3 We changed the optimizer to Adam and raised the batch size to 32 in order to enhance convergence and model stability. The purpose of this experiment was to determine whether a different optimizer and a higher batch size may improve the model's generalization.
  The Results showed that there were some improvement in the metrics: mAP50-95: 0.53251 mAP50: 0.62099. Although the model's performance improved somewhat, overfitting and mislabeling problems remained. Images with ruled or boxed backgrounds and closely spaced

digits continued to cause problems for the bounding box predictions.

Even though training stability was enhanced by the larger batch size, the model was still unable to converge successfully on difficult data. In order to further counteract overfitting, early halting and data augmentation were decided to be implemented.

- Experiment 4

  To test if these parameters would result in better generalization and enhanced performance over earlier setups, we further expanded the training setup by adding an AdamW optimizer, a batch size of 32, and an image size of 640px. Results: Although the model made some minor progress, overfitting problems persisted. Furthermore, the issue of mislabeling bounding boxes and the inability to manage ruled backgrounds was not entirely fixed as could be visually seen from the predicted boxes.

  Conclusion: The experiment's findings demonstrated the need for additional tactics, including early stopping and data augmentation, to lessen overfitting and increase model robustness, even in spite of the slight performance improvements.
- Experiment 5

  To lessen overfitting and improve model performance, we enabled data augmentation and implemented early halting (with a 10-epoch patience) in this last trial. A batch size of 32, a learning rate of 0.001, and an image size of 640 pixels were used to train the model. Results: Across all criteria, this trial produced the best results. mAP50-95: 0.5551 mAP50: 0.63487. Early stopping was included to avoid overfitting and enable the model to cease training when performance reached a plateau. In order to ensure that the model viewed a wider variety of images and enhance generalization and performance on unseen data, data augmentation increased the heterogeneity of the training data. The model performed better on photographs with boxed or ruled backgrounds, and bounding box problems were significantly decreased.

  With greater mAP scores and improved handling of difficult input images, this trial yielded the best-trained model. A resilient model that could more effectively generalize to new data was produced by combining early halting, data augmentation, and modified hyperparameters (such as batch size and optimizer).

In the initial experiments, bounding box problems were most noticeable, such as several boxes surrounding closely spaced numbers or inaccurate predictions on pictures with ruled backgrounds. These problems were probably brought about by the model's inability to discriminate between background patterns and digits, especially when the digits were positioned closely together. Furthermore, the model was improved by using early halting, which stopped it from learning incorrect predictions.

On seeing the bounding box predictions on the train dataset, we could visually make out that the majority models were unable to detect digits on lined, ruled or boxed pages, but was converging well for blank page digits. Similarly, some of the models were unable to detect images which were on blank pages, but could predict better bounding boxes on the ruled or boxed pages. To mitigate these, the 5th model had the best bounding box predictions, with high confidence level for all the class labels and a good confusion matrix to back it up.

V. Conclusion

This project was aimed at developing, with the use of YOLOv8 transfer learning, an effective model for recognizing handwritten digits. We investigate several ways of improving the performance of this model through five carefully designed experiments that study various hyperparameters, training strategies, and techniques to address such specific challenges as overfitting and mislabeling of bounding boxes, especially for closely placed digits or against ruled or boxed backgrounds.

Early stopping with patience of 10 epochs was an important step in reducing overfitting. It helped the model stop training when performance plateaued on the validation set, hence preventing it from memorizing noise or irrelevant patterns in the training data. Data augmentation further improved generalization by providing a more diverse range of images. These significantly enhanced the robustness of the models, especially in Experiment 5, where these techniques helped achieve the best performance.

The models of experiments 1 and 2, which had not used early stopping and data augmentation, respectively, clearly showed signs of overfitting, especially when digits are close to each other or the picture has background noise, such as ruled lines. This is confirmed by their generalization on test data that showed the importance of those techniques in avoiding overfitting.

Also, the model showed robust generalization across the validation data such that certain digits like '1' and '7' have always been detected quite well. Overlapping digits create problems, especially for digits '5' and '8', reducing the detection scores for these classes. While increasing epochs did improve performance somewhat, the experiments showed that additional epochs are not enough to create significant gains without appropriate settings of batch size and optimizer.

The performance could be further improved by incorporating systematic data augmentation that can improve model robustness and deal with issues of class imbalance. The project showed that hyperparameter tuning was important in realizing the best performance. In this case, batch size and optimizer choice had more profound impacts than the number of epochs alone.

VI. References

1) .N. Annapareddy et al., "Handwritten Text and Digit Classification on Rwandan Perioperative Flowsheets via YOLOv5," 2022 Systems and Information Engineering Design Symposium (SIEDS), Charlottesville, VA, USA, 2022, pp. 270-275, doi:

10.1109/SIEDS55548.2022.9799426. keywords: Text categorization;Neural networks;Surgery;Object detection;Manuals;Physiology;Trajectory,

2) .H. A. Raboanary, R. Raboanary and N. H. M. Tahiridimbisoa, ”Robustness Assessment of Neural Network Architectures to Geometric Transformations: A Comparative Study with Data Augmentation,” 2023 3rd International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME), Tenerife, Canary Islands, Spain, 2023, pp. 1-6, doi: 10.1109/ICECCME57830.2023.10253075. keywords: Adaptation models;Neural networks;Transfer learning;Computer architecture;Data augmentation;Transformers;Robustness;Deep learning;Invariance;Data augmentation;Transfer learning,
3) .W. Zhuang, X. -G. Tang, G. Yang, G. Yuan and H. Yu, ”Remote sensing image object detection based on rotatable bounding box,” 2021 International Conference on Digital Society and Intelligent Systems (DSInS), Chengdu, China, 2021, pp. 172-177, doi: 10.1109/DSInS54396.2021.9670625. keywords: Training;Photography;Smoothing methods;Target recognition;Object detection;Real-time systems;Classification algorithms;object detection;remote sensing image;YOLOv5;rotatable bounding box,